Client-Server & Multi-Tier Architecture

why almost every app is split into layers

Watch the 3-min video

In one line: a client asks, a server answers; splitting the server side into presentation, application and data tiers lets each part scale, change and be secured on its own.

1 Client and server

A **client** (a browser, a mobile app, another service) sends a **request**. A **server** does the work and sends back a **response**. The client never touches the server's internals; it only sees the API. That contract is what lets you change, scale or replace the server without shipping a new client.

2 One, two, three … N tiers

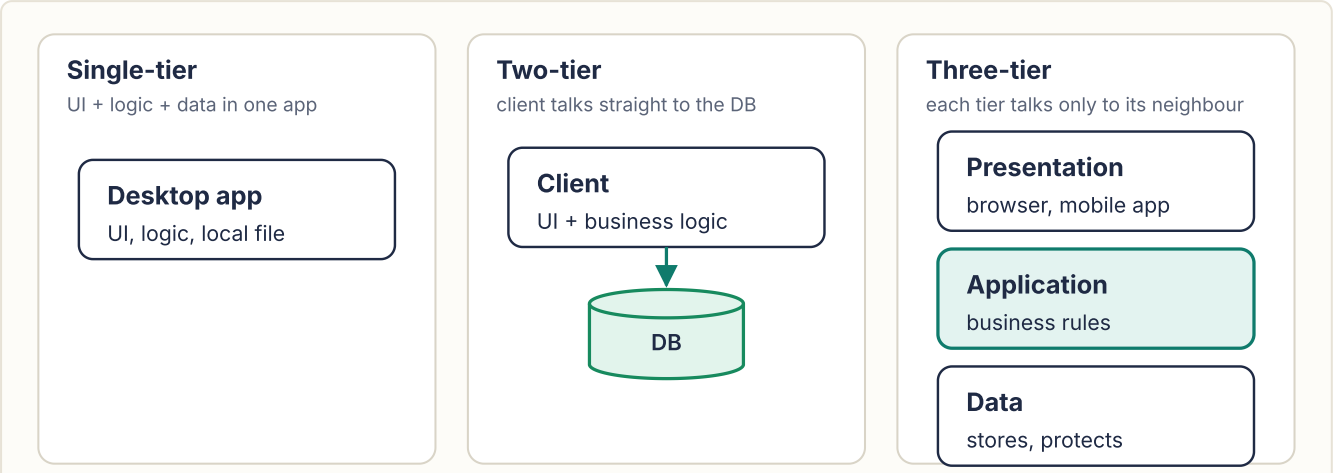
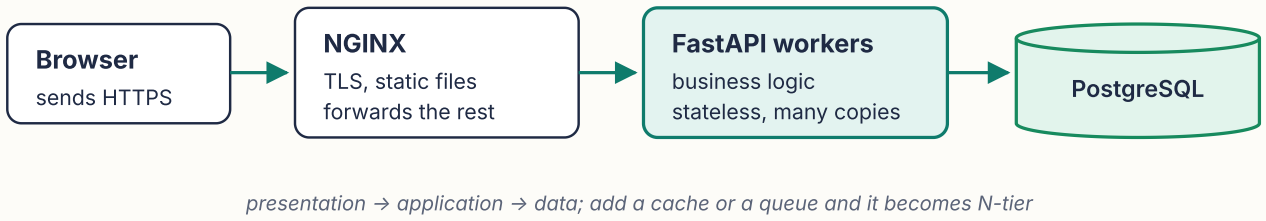


Figure 1. As tiers are added, business logic moves out of the client and the database stops being exposed to every user.

Layout	Where the logic lives	Good for	Problem
Single-tier	One program on one machine	Desktop tools, scripts	Nothing is shared or scalable
Two-tier	In every client	Small internal tools	DB exposed to all clients; logic copied everywhere
Three-tier	The application tier	Most web and mobile apps	More moving parts to run
N-tier	App tier + caches, queues, workers	Large systems	Operational complexity, more hops

3 A real request, tier by tier



presentation → application → data; add a cache or a queue and it becomes N-tier

Figure 2. A typical three-tier web stack. Only the application tier can reach the database.

1. **Browser → NGINX.** The reverse proxy terminates TLS, serves static files and forwards API calls.
2. **NGINX → FastAPI workers.** A pool of identical, stateless workers runs the business rules.
3. **Workers → PostgreSQL.** Reads and writes go to the data tier, which owns the state.

4 Why separate the tiers

- **Independent scaling.** The app tier is stateless, so you scale it *out* by adding workers. The data tier holds state, so it scales differently: a bigger machine, read replicas, or sharding.
- **Independent change.** Redesign the UI without touching the database; migrate the database without changing the UI.
- **Security.** Only the app tier can reach the database, so the data sits behind one controlled door instead of being open to every client.

WORKED EXAMPLE: THE SLOWEST TIER WINS

The database is overloaded and every page slows to 2 s. You double the app servers. Pages get *slower*: twice the workers open twice the database connections and add load to the bottleneck. The fix belongs in the data tier: a cache for hot reads, connection pooling, read replicas, or better indexes.

TRADE-OFF: MORE TIERS VS FEWER HOPS

Every tier adds a network hop (often 0.5–2 ms inside a data centre), another thing to deploy and another thing that can fail. Add a tier when it solves a real problem (scaling, security, caching), not by default.

COMMON MISTAKES

- Scaling the app tier when the database is the bottleneck.
- Keeping user sessions in an app server's memory, which breaks horizontal scaling.
- Letting clients talk to the database directly “just for this report”.

5 Key terms

Client / server: asks / answers

Presentation tier: what users see

Data tier: owns and protects state

Tier: a layer with one job, often its own machines

Application tier: business rules, stateless

Reverse proxy: front door: TLS, routing, static files

6 Interview questions

Q1. Why separate the application tier from the data tier? BASIC

So each can scale, change and be secured independently: stateless app servers scale out cheaply, the stateful database scales with replicas or sharding, and only the app tier can reach the data.

Q2. What's the difference between two-tier and three-tier? BASIC

In two-tier, clients talk to the database directly and carry the business logic. Three-tier puts an application server in between, so logic lives in one place and the database is no longer exposed.

Q3. Why must the application tier be stateless to scale out? BASIC

Any copy must be able to serve any request. If a session lives in one server's memory, a load balancer can't move that user elsewhere; keep state in a database, cache or signed token instead.

Q4. Pages slow down when the database is overloaded. You add app servers. What happens? SCENARIO

It gets worse: more app servers open more database connections and add load to the bottleneck. Measure, then fix the data tier with caching, pooling, read replicas or indexing.

Q5. Where would you put a cache in a three-tier app? SCENARIO

Between the application tier and the database for hot reads (Redis), and at the edge for static content (CDN). The app tier checks the cache first and falls back to the database.

Q6. A mobile app needs data from three backend services. How do you avoid three round trips from the phone? SCENARIO

Add a layer in front (an API gateway or a backend-for-frontend) that calls the services in parallel inside the data centre and returns one response.

Q7. Every tier adds latency. When is a new tier not worth it? TRICKY

When it doesn't solve a measured problem. Each hop adds network time, deployment work and a failure point; a small app with low traffic is often better as a simple three-tier stack.

Q8. Is a microservices system still “three-tier”? TRICKY

Each service is usually its own small multi-tier stack (API, logic, its own database). The system overall is N-tier: gateway, many services, caches, queues and data stores.

Next: SD-A-003 · Functional Requirements backendroadmap.com